

Beyond A Better Planning with Transformers via Search Dynamics Bootstrapping

One-line Summary

The main idea is instead of just prompt->plan paper focuses on improving performance by training Transformer on A* search traces + plan.

They also introduce Search Dynamics Bootstrapping, which iteratively discovers shorter successful traces and is retrained on these shorter traces to improve efficiency.

Problem

Traditional planning approaches learn Problem -> Solution and ignores the process used to obtain the solution.

Does learning search traces improve planning generalisation compared to learning only final solutions?

Core Idea

Train the model on:

Problem → Search Trace → Solution

The Search Trace is obtained using A* and contains details about how the nodes were explored during the plan generation.

The hypothesis is that search traces provide richer supervision than final plans alone.

Key Concepts

Search-Augmented Model

Model trained on both search traces and plans.

Search Dynamics

Internal operations of A* such as:

- Create node
- Close node
- Heuristic values
- Cost values

Non-Deterministic A* /

Random tie-breaking and child ordering generate multiple valid traces for the same problem.

Search Dynamics Bootstrapping (SDB)

Model-generated traces replace original traces if they are:

- Valid
- Optimal
- Shorter

- The model is then retrained on these improved traces.

Searchformer

Final model obtained after multiple bootstrapping iterations.

Method pipeline

1. Generate Search Trees

Problem -> A* -> Search Trace + Optimal Plan

Solution-only sequences of the format (prompt)(plan)

Search-augmented sequences of the format (prompt)(trace)(plan)

2. Train Search- Augmented Transformer

Problem -> Transformer -> Search Trace + Plan

For each experiment an adaptation of the encoder-decoder T5 architecture is used that integrates Rotary Position Embeddings (RoPE)

3. Search Dynamics Bootstrapping

Current model -> generate multiple traces -> keep valid and shorter traces -> replace training data -> retrain

Repeat this for multiple iterations.

Results

Maze Navigation

- Search-augmented models consistently outperform solution-only models.
- As maze size increases the solution only model's performance drops very sharply.
- Improvements persist even when A* traces are non-deterministic.

Sokoban

- Search-augmented models outperform solution-only models.
- A 175M search-augmented model outperforms a much larger 747M solution-only model.
- Searchformer achieves **93.7% optimal solution rate**.

Bootstrapping

- Successive bootstrapping iterations produce shorter search traces.

- Improvements occur across the entire trace-length distribution, not only for a few examples.
- Trace efficiency improves while maintaining solution quality.

Weaknesses

- Limited Domains : Only restricted to maze navigation and Sokoban
- As maze size increases performance drops so not generalized very well.
- trace length may grow exponentially in the length of an optimal plan and training on the resulting token sequence data can become computationally very expensive

My thoughts

- How can generalise by analysing on only 2 domains may be this works only for these and fails in rest.
- I appreciate the idea of reasoning should be learned and not hard-coded as different problems need different reasoning.

Connections

Chain-of-Thought (CoT):

CoT: Problem → Human Reasoning → Answer

Searchformer: Problem → Algorithmic Search Trace → Answer

AlphaZero

This uses search as a policy improvement operator but does not train directly on search traces.

AlphaZero: Problem → Search → Better Action

Searchformer: Problem → Search Trace → Learn Search Behavior